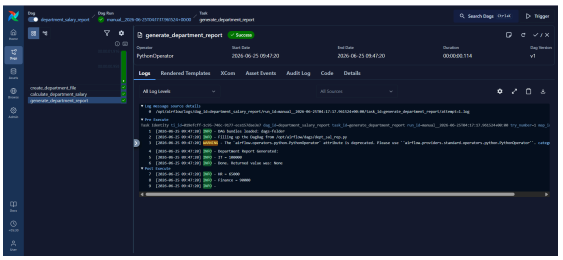
ex07.py

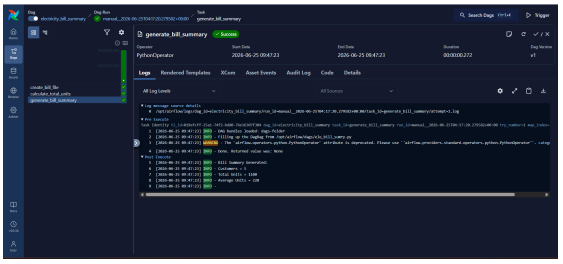
```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime


# Task 1: Create input file
def create_department_file():
    data = """IT,45000
HR,35000
Finance,50000
IT,55000
Finance,40000
HR,30000"""

    with open('/tmp/departments.txt', 'w') as f:
        f.write(data)

    print("Department file created successfully.")


# Task 2: Calculate total salary department-wise
def calculate_department_salary(ti):
    department_totals = {}

    with open('/tmp/departments.txt', 'r') as f:
        for line in f:
            dept, salary = line.strip().split(',')
            salary = int(salary)

            if dept in department_totals:
                department_totals[dept] += salary
            else:
                department_totals[dept] = salary

    print(department_totals)

    # Store result using XCom
    ti.xcom_push(key='dept_salary',
value=department_totals)


# Task 3: Generate report file
def generate_department_report(ti):
    department_totals = ti.xcom_pull(
        task_ids='calculate_department_salary',
        key='dept_salary'
    )

    with open('/tmp/department_report.txt', 'w') as f:
        for dept, total in department_totals.items():
            f.write(f"{dept} = {total}\n")

    print("Department Report Generated:")
    with open('/tmp/department_report.txt', 'r') as f:
        print(f.read())


# DAG Definition
with DAG(
    dag_id='department_salary_report',
    start_date=datetime(2025, 1, 1),
    schedule=None,
    catchup=False,
) as dag:

    create_department_file_task = PythonOperator(
        task_id='create_department_file',
        python_callable=create_department_file
    )

    calculate_department_salary_task = PythonOperator(
        task_id='calculate_department_salary',
        python_callable=calculate_department_salary
    )

    generate_department_report_task = PythonOperator(
        task_id='generate_department_report',
        python_callable=generate_department_report
    )

    # Task Dependencies
    create_department_file_task >>
calculate_department_salary_task >>
generate_department_report_task
```



ex08.py

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime


# Task 1: Create electricity file
def create_bill_file():
    data = """Rahul,210
Priya,180
Amit,300
Sneha,150
Kiran,260"""
```

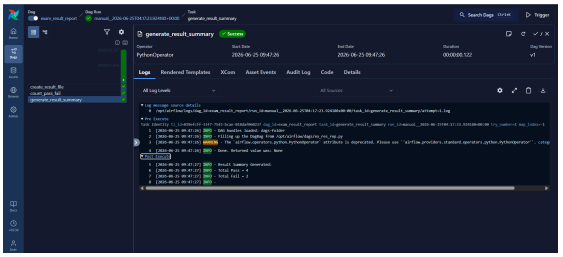
```
    with open('/tmp/electricity.txt', 'w') as f:
        f.write(data)

    print("Electricity file created successfully.")


# Task 2: Calculate total and average units
def calculate_total_units(ti):
    total_units = 0
    customer_count = 0

    with open('/tmp/electricity.txt', 'r') as f:
        for line in f:
            customer, units = line.strip().split(',')
            total_units += int(units)
            customer_count += 1

    average_units = total_units / customer_count

    print(f"Total Units: {total_units}")
    print(f"Average Units: {average_units}")

    # Store values using XCom
    ti.xcom_push(key='total_units', value=total_units)
    ti.xcom_push(key='average_units',
value=average_units)
    ti.xcom_push(key='customer_count',
value=customer_count)


# Task 3: Generate summary report
def generate_bill_summary(ti):
    total_units = ti.xcom_pull(
        task_ids='calculate_total_units',
        key='total_units'
    )

    average_units = ti.xcom_pull(
        task_ids='calculate_total_units',
        key='average_units'
    )

    customer_count = ti.xcom_pull(
        task_ids='calculate_total_units',
        key='customer_count'
    )

    with open('/tmp/bill_summary.txt', 'w') as f:
        f.write(f"Customers = {customer_count}\n")
        f.write(f"Total Units = {total_units}\n")
        f.write(f"Average Units = {average_units}\n")

    print("Bill Summary Generated:")

    with open('/tmp/bill_summary.txt', 'r') as f:
        print(f.read())


# DAG Definition
with DAG(
    dag_id='electricity_bill_summary',
    start_date=datetime(2025, 1, 1),
    schedule=None,
    catchup=False,
) as dag:

    create_bill_file_task = PythonOperator(
        task_id='create_bill_file',
        python_callable=create_bill_file
    )

    calculate_total_units_task = PythonOperator(
        task_id='calculate_total_units',
        python_callable=calculate_total_units
    )

    generate_bill_summary_task = PythonOperator(
        task_id='generate_bill_summary',
        python_callable=generate_bill_summary
    )

    # Task Dependencies
    create_bill_file_task >> calculate_total_units_task >>
generate_bill_summary_task
```



ex09.py

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime


# Task 1: Create results file
def create_result_file():
    data = """Rahul,Pass
Priya,Fail
Amit,Pass
Sneha,Pass
Kiran,Fail
Megha,Pass"""

    with open('/tmp/results.txt', 'w') as f:
```

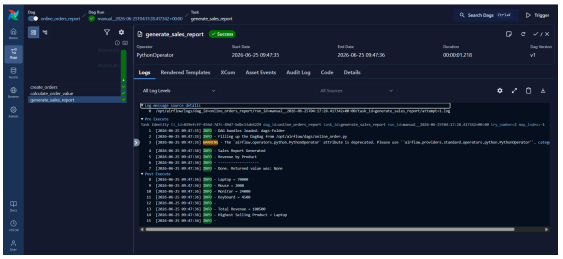
```
        f.write(data)

    print("Result file created successfully.")


# Task 2: Count pass and fail students
def count_pass_fail(ti):
    pass_count = 0
    fail_count = 0

    with open('/tmp/results.txt', 'r') as f:
        for line in f:
            student, result = line.strip().split(',')

            if result == 'Pass':
                pass_count += 1
            else:
                fail_count += 1

    print(f"Total Pass: {pass_count}")
    print(f"Total Fail: {fail_count}")

    # Store values in XCom
    ti.xcom_push(key='pass_count', value=pass_count)
    ti.xcom_push(key='fail_count', value=fail_count)


# Task 3: Generate summary report
def generate_result_summary(ti):
    pass_count = ti.xcom_pull(
        task_ids='count_pass_fail',
        key='pass_count'
    )

    fail_count = ti.xcom_pull(
        task_ids='count_pass_fail',
        key='fail_count'
    )

    with open('/tmp/result_summary.txt', 'w') as f:
        f.write(f"Total Pass = {pass_count}\n")
        f.write(f"Total Fail = {fail_count}\n")

    print("Result Summary Generated:")
    with open('/tmp/result_summary.txt', 'r') as f:
        print(f.read())


# DAG Definition
with DAG(
    dag_id='exam_result_report',
    start_date=datetime(2025, 1, 1),
    schedule=None,
    catchup=False,
) as dag:

    create_result_file_task = PythonOperator(
        task_id='create_result_file',
        python_callable=create_result_file
    )

    count_pass_fail_task = PythonOperator(
        task_id='count_pass_fail',
        python_callable=count_pass_fail
    )

    generate_result_summary_task = PythonOperator(
        task_id='generate_result_summary',
        python_callable=generate_result_summary
    )

    # Task Dependencies
    create_result_file_task >> count_pass_fail_task >>
generate_result_summary_task
```



[ex10.py](#)

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
import csv


# Task 1: Create orders CSV file
def create_orders():
    with open('/tmp/orders.csv', 'w', newline='') as f:
        writer = csv.writer(f)

        # Header
        writer.writerow(['product', 'quantity', 'price'])

        # Data
        writer.writerow(['Laptop', 1, 70000])
        writer.writerow(['Mouse', 4, 500])
        writer.writerow(['Monitor', 2, 12000])
        writer.writerow(['Keyboard', 3, 1500])

    print("orders.csv created successfully.")


# Task 2: Calculate revenue for each product
```

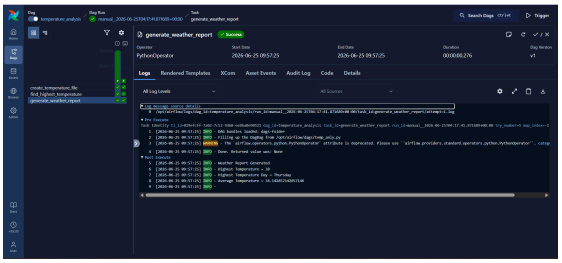
```
def calculate_order_value(ti):
    total_revenue = 0
    revenues = {}

    with open('/tmp/orders.csv', 'r') as f:
        reader = csv.DictReader(f)

        for row in reader:
            product = row['product']
            quantity = int(row['quantity'])
            price = int(row['price'])

            revenue = quantity * price
            revenues[product] = revenue

            total_revenue += revenue

        # Highest selling product based on revenue
        highest_product = max(revenues, key=revenues.get)

    print(revenues)
    print(f"Total Revenue: {total_revenue}")
    print(f"Highest Selling Product: {highest_product}")

    # Push values to XCom
    ti.xcom_push(key='revenues', value=revenues)
    ti.xcom_push(key='total_revenue',
value=total_revenue)
    ti.xcom_push(key='highest_product',
value=highest_product)


# Task 3: Generate sales report
def generate_sales_report(ti):
    revenues = ti.xcom_pull(
        task_ids='calculate_order_value',
        key='revenues'
    )

    total_revenue = ti.xcom_pull(
        task_ids='calculate_order_value',
        key='total_revenue'
    )

    highest_product = ti.xcom_pull(
        task_ids='calculate_order_value',
        key='highest_product'
    )

    with open('/tmp/sales_report.txt', 'w') as f:

        f.write("Revenue by Product\n")
        f.write("-------------------\n")
        for product, revenue in revenues.items():
            f.write(f"{product} = {revenue}\n")

        f.write("\n")
        f.write(f"Total Revenue = {total_revenue}\n")
        f.write(f"Highest Selling Product =
{highest_product}\n")

    print("Sales Report Generated")

    with open('/tmp/sales_report.txt', 'r') as f:
        print(f.read())


# DAG Definition
with DAG(
    dag_id='online_orders_report',
    start_date=datetime(2025, 1, 1),
    schedule=None,
    catchup=False
) as dag:

    create_orders_task = PythonOperator(
        task_id='create_orders',
        python_callable=create_orders
    )

    calculate_order_value_task = PythonOperator(
        task_id='calculate_order_value',
        python_callable=calculate_order_value
    )

    generate_sales_report_task = PythonOperator(
        task_id='generate_sales_report',
        python_callable=generate_sales_report
    )

    # Task Dependencies
    create_orders_task >> calculate_order_value_task >>
generate_sales_report_task
```



ex11.py

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
```

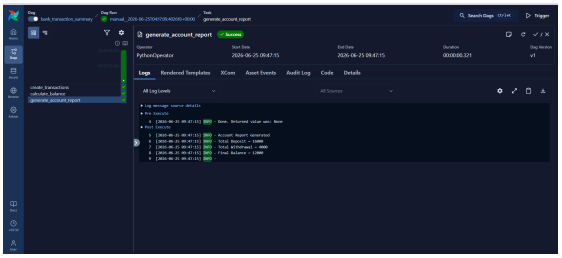
```
# Task 1: Create temperature file
def create_temperature_file():
    data = """Monday,34
Tuesday,36
Wednesday,31
Thursday,38
Friday,35
Saturday,33
Sunday,32"""

    with open('/tmp/temperature.txt', 'w') as f:
        f.write(data)

    print("Temperature file created successfully.")


# Task 2: Find highest and average temperature
def find_highest_temperature(ti):
    temperatures = []
    highest_day = ""
    highest_temp = 0

    with open('/tmp/temperature.txt', 'r') as f:
        for line in f:
            day, temp = line.strip().split(',')
            temp = int(temp)

            temperatures.append(temp)

            if temp > highest_temp:
                highest_temp = temp
                highest_day = day

    average_temp = sum(temperatures) /
len(temperatures)

    print(f"Highest Temperature: {highest_temp}")
    print(f"Average Temperature: {average_temp}")

    # Push values to XCom
    ti.xcom_push(key='highest_temp',
value=highest_temp)
    ti.xcom_push(key='highest_day', value=highest_day)
    ti.xcom_push(key='average_temp',
value=average_temp)


# Task 3: Generate weather report
def generate_weather_report(ti):
    highest_temp = ti.xcom_pull(
        task_ids='find_highest_temperature',
        key='highest_temp'
    )

    highest_day = ti.xcom_pull(
        task_ids='find_highest_temperature',
        key='highest_day'
    )

    average_temp = ti.xcom_pull(
        task_ids='find_highest_temperature',
        key='average_temp'
    )

    with open('/tmp/weather_report.txt', 'w') as f:
        f.write(f"Highest Temperature = {highest_temp}\n")
        f.write(f"Highest Temperature Day =
{highest_day}\n")
        f.write(f"Average Temperature = {average_temp}\n")

    print("Weather Report Generated")

    with open('/tmp/weather_report.txt', 'r') as f:
        print(f.read())


# DAG Definition
with DAG(
    dag_id='temperature_analysis',
    start_date=datetime(2025, 1, 1),
    schedule=None,
    catchup=False
) as dag:

    create_temperature_file_task = PythonOperator(
        task_id='create_temperature_file',
        python_callable=create_temperature_file
    )

    find_highest_temperature_task = PythonOperator(
        task_id='find_highest_temperature',
        python_callable=find_highest_temperature
    )

    generate_weather_report_task = PythonOperator(
        task_id='generate_weather_report',
        python_callable=generate_weather_report
    )

    # Task Dependencies
    create_temperature_file_task >> \
    find_highest_temperature_task >> \
    generate_weather_report_task
```



ex12.py

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime


# Task 1: Create transactions file
def create_transactions():
    data = """Deposit,10000
Withdraw,2500
Deposit,4000
Withdraw,1500
Deposit,2000"""

    with open('/tmp/transactions.txt', 'w') as f:
        f.write(data)

    print("Transactions file created successfully.")


# Task 2: Calculate deposits, withdrawals, and balance
def calculate_balance(ti):
    total_deposit = 0
    total_withdrawal = 0

    with open('/tmp/transactions.txt', 'r') as f:
        for line in f:
            transaction_type, amount = line.strip().split(',')
            amount = int(amount)

            if transaction_type == 'Deposit':
                total_deposit += amount
            elif transaction_type == 'Withdraw':
                total_withdrawal += amount

    final_balance = total_deposit - total_withdrawal

    print(f"Total Deposit: {total_deposit}")
    print(f"Total Withdrawal: {total_withdrawal}")
    print(f"Final Balance: {final_balance}")

    # Push values to XCom
    ti.xcom_push(key='total_deposit', value=total_deposit)
    ti.xcom_push(key='total_withdrawal',
value=total_withdrawal)
    ti.xcom_push(key='final_balance',
value=final_balance)


# Task 3: Generate account report
def generate_account_report(ti):
    total_deposit = ti.xcom_pull(
        task_ids='calculate_balance',
        key='total_deposit'
    )

    total_withdrawal = ti.xcom_pull(
        task_ids='calculate_balance',
        key='total_withdrawal'
    )

    final_balance = ti.xcom_pull(
        task_ids='calculate_balance',
        key='final_balance'
    )

    with open('/tmp/account_report.txt', 'w') as f:
        f.write(f"Total Deposit = {total_deposit}\n")
        f.write(f"Total Withdrawal = {total_withdrawal}\n")
        f.write(f"Final Balance = {final_balance}\n")

    print("Account Report Generated")

    with open('/tmp/account_report.txt', 'r') as f:
        print(f.read())


# DAG Definition
with DAG(
    dag_id='bank_transaction_summary',
    start_date=datetime(2025, 1, 1),
    schedule=None,
    catchup=False
) as dag:

    create_transactions_task = PythonOperator(
        task_id='create_transactions',
        python_callable=create_transactions
    )

    calculate_balance_task = PythonOperator(
        task_id='calculate_balance',
        python_callable=calculate_balance
    )

    generate_account_report_task = PythonOperator(
        task_id='generate_account_report',
        python_callable=generate_account_report
```

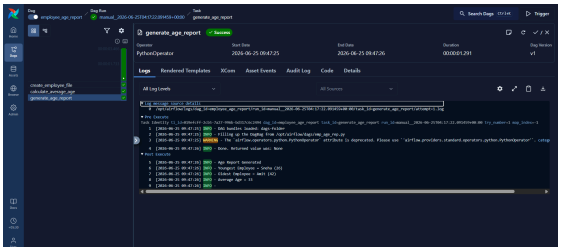
```
    )

    # Task Dependencies
    create_transactions_task >> calculate_balance_task
>> generate_account_report_task
```



[ex13.py](#)

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime


# Task 1: Create employee file
def create_employee_file():
    data = """Rahul,28
Priya,31
Amit,42
Sneha,26
Kiran,38"""

    with open('/tmp/employees.txt', 'w') as f:
        f.write(data)

    print("Employee file created successfully.")


# Task 2: Calculate youngest, oldest, and average age
def calculate_average_age(ti):
    total_age = 0
    employee_count = 0

    youngest_employee = ""
    youngest_age = float('inf')

    oldest_employee = ""
    oldest_age = 0

    with open('/tmp/employees.txt', 'r') as f:
        for line in f:
            employee, age = line.strip().split(',')
            age = int(age)

            total_age += age
            employee_count += 1

            if age < youngest_age:
                youngest_age = age
                youngest_employee = employee

            if age > oldest_age:
                oldest_age = age
                oldest_employee = employee

    average_age = total_age / employee_count

    print(f"Youngest Employee: {youngest_employee}
({youngest_age})")
    print(f"Oldest Employee: {oldest_employee}
({oldest_age})")
    print(f"Average Age: {average_age}")

    # Push values to XCom
    ti.xcom_push(key='youngest_employee',
value=youngest_employee)
    ti.xcom_push(key='youngest_age',
value=youngest_age)

    ti.xcom_push(key='oldest_employee',
value=oldest_employee)
    ti.xcom_push(key='oldest_age', value=oldest_age)

    ti.xcom_push(key='average_age', value=average_age)


# Task 3: Generate age report
def generate_age_report(ti):
    youngest_employee = ti.xcom_pull(
        task_ids='calculate_average_age',
        key='youngest_employee'
    )

    youngest_age = ti.xcom_pull(
        task_ids='calculate_average_age',
        key='youngest_age'
    )

    oldest_employee = ti.xcom_pull(
        task_ids='calculate_average_age',
        key='oldest_employee'
    )

    oldest_age = ti.xcom_pull(
        task_ids='calculate_average_age',
        key='oldest_age'
    )

    average_age = ti.xcom_pull(
        task_ids='calculate_average_age',
        key='average_age'
    )
```

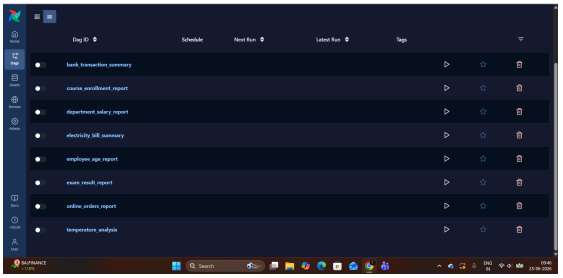
```
    with open('/tmp/age_report.txt', 'w') as f:
        f.write(f"Youngest Employee = {youngest_employee}
({youngest_age})\n")
        f.write(f"Oldest Employee = {oldest_employee}
({oldest_age})\n")
        f.write(f"Average Age = {average_age}\n")

    print("Age Report Generated")

    with open('/tmp/age_report.txt', 'r') as f:
        print(f.read())


# DAG Definition
with DAG(
    dag_id='employee_age_report',
    start_date=datetime(2025, 1, 1),
    schedule=None,
    catchup=False
) as dag:

    create_employee_file_task = PythonOperator(
        task_id='create_employee_file',
        python_callable=create_employee_file
    )

    calculate_average_age_task = PythonOperator(
        task_id='calculate_average_age',
        python_callable=calculate_average_age
    )

    generate_age_report_task = PythonOperator(
        task_id='generate_age_report',
        python_callable=generate_age_report
    )

    # Task Dependencies
    create_employee_file_task >> \
    calculate_average_age_task >> \
    generate_age_report_task
```

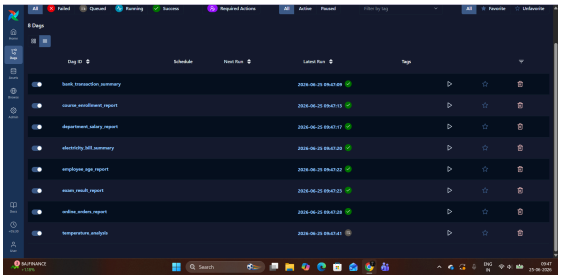


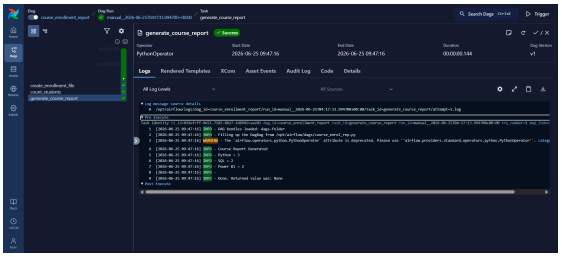
ex14.py

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

# Task 1: Create enrollment file
def create_enrollment_file():
    data = """Python,Rahul
Python,Priya
SQL,Amit
Python,Sneha
Power BI,Kiran
SQL,Megha
Power BI,Arjun"""

    with open('/tmp/enrollments.txt', 'w') as f:
        f.write(data)

    print("Enrollment file created successfully.")


# Task 2: Count students in each course
def count_students(ti):
    course_counts = {}

    with open('/tmp/enrollments.txt', 'r') as f:
        for line in f:
            course, student = line.strip().split(',')

            if course in course_counts:
                course_counts[course] += 1
            else:
                course_counts[course] = 1

    print(course_counts)

    # Store result in XCom
    ti.xcom_push(key='course_counts',
value=course_counts)


# Task 3: Generate course report
def generate_course_report(ti):
    course_counts = ti.xcom_pull(
        task_ids='count_students',
        key='course_counts'
    )

    with open('/tmp/course_report.txt', 'w') as f:
        for course, count in course_counts.items():
            f.write(f"{course} = {count}\n")

    print("Course Report Generated")

    with open('/tmp/course_report.txt', 'r') as f:
        print(f.read())
```

```
# DAG Definition
with DAG(
    dag_id='course_enrollment_report',
    start_date=datetime(2025, 1, 1),
    schedule=None,
    catchup=False
) as dag:

    create_enrollment_file_task = PythonOperator(
        task_id='create_enrollment_file',
        python_callable=create_enrollment_file
    )

    count_students_task = PythonOperator(
        task_id='count_students',
        python_callable=count_students
    )

    generate_course_report_task = PythonOperator(
        task_id='generate_course_report',
        python_callable=generate_course_report
    )

    # Task Dependencies
    create_enrollment_file_task >> \
    count_students_task >> \
    generate_course_report_task
```







```
root@ubuntu:~$ docker run -d -p 8080:8080 --name airflow apache/airflow standalone
Unable to find image 'apache/airflow:latest' locally
latest: Pulling from apache/airflow
068fedd6b0f1: Pull complete
```

Simple auth manager | Password for user 'admin': GSvVdr7DbcuuCXVB

```
airflow@710cbebc30e9:/opt/airflow/dags$ airflow dags list-import-errors
2026-06-25T04:15:13.817603Z [info     ] setup plugin alembic.autogenerate.schemas [alem
2026-06-25T04:15:13.818475Z [info     ] setup plugin alembic.autogenerate.tables [alemb
2026-06-25T04:15:13.819023Z [info     ] setup plugin alembic.autogenerate.types [alembi
2026-06-25T04:15:13.819608Z [info     ] setup plugin alembic.autogenerate.constraints [
2026-06-25T04:15:13.820097Z [info     ] setup plugin alembic.autogenerate.defaults [ale
2026-06-25T04:15:13.820537Z [info     ] setup plugin alembic.autogenerate.comments [ale
No data found
airflow@710cbebc30e9:/opt/airflow/dags$ airflow dags reserialize
2026-06-25T04:15:25.615004Z [info     ] setup plugin alembic.autogenerate.schemas [alem
2026-06-25T04:15:25.615540Z [info     ] setup plugin alembic.autogenerate.tables [alemb
2026-06-25T04:15:25.616031Z [info     ] setup plugin alembic.autogenerate.types [alembi
2026-06-25T04:15:25.616507Z [info     ] setup plugin alembic.autogenerate.constraints [
```

```
airflow@710cbebc30e9:/opt/airflow/dags$ airflow dags list
2026-06-25T04:15:39.753787Z [info     ] setup plugin alembic.autogenerate.schemas [alembic.runtime.plugins] loc=plugins.py:37
2026-06-25T04:15:39.754158Z [info     ] setup plugin alembic.autogenerate.tables [alembic.runtime.plugins] loc=plugins.py:37
2026-06-25T04:15:39.754853Z [info     ] setup plugin alembic.autogenerate.types [alembic.runtime.plugins] loc=plugins.py:37
2026-06-25T04:15:39.755374Z [info     ] setup plugin alembic.autogenerate.constraints [alembic.runtime.plugins] loc=plugins.py:37
2026-06-25T04:15:39.755898Z [info     ] setup plugin alembic.autogenerate.defaults [alembic.runtime.plugins] loc=plugins.py:37
2026-06-25T04:15:39.756580Z [info     ] setup plugin alembic.autogenerate.comments [alembic.runtime.plugins] loc=plugins.py:37
dag_id                | fileloc                                          | owners  | is_paused | bundle_name | bundle_version
======================+==================================================+=========+===========+=============+===============
bank_transaction_summary | /opt/airflow/dags/bank_tran_sumry.py          | airflow | True      | dags-folder | None
course_enrollment_report | /opt/airflow/dags/course_enrol_rep.py         | airflow | True      | dags-folder | None
department_salary_report | /opt/airflow/dags/dept_sal_rep.py             | airflow | True      | dags-folder | None
electricity_bill_summary | /opt/airflow/dags/ele_bill_sumry.py           | airflow | True      | dags-folder | None
employee_age_report      | /opt/airflow/dags/emp_age_rep.py              | airflow | True      | dags-folder | None
exam_result_report       | /opt/airflow/dags/ex_res_rep.py               | airflow | True      | dags-folder | None
online_orders_report     | /opt/airflow/dags/online_order.py             | airflow | True      | dags-folder | None
temperature_analysis     | /opt/airflow/dags/temp_anly.py                | airflow | True      | dags-folder | None
```